Rockchip MIPI DSI Interface Multi-Panel Compatibility Developer Guide

ID: RK-KF-YF-E26

Release Version: V1.1.0

Release Date: 2025-05-16

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Product Version

Chip Models	U-Boot Version
RK3288	next-dev
RK3399	next-dev
RK3326/PX30	next-dev
RK356x	next-dev
RK3588	next-dev
RK3576	next-dev

Intended Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Description
V1.0.0	Huang Guochun	2025-03-06	Initial release
V1.1.0	Huang Guochun	2025-05-16	Fixed LPDT communication anomaly after obtaining panel ID

Table of Contents

Rockchip MIPI DSI Interface Multi-Panel Compatibility Developer Guide

1. Introduction
2. Core Functionality Description
   - 2.1 Multi-Display Compatibility Mechanism
     - 2.1.1 Dynamic Panel Detection
     - 2.1.2 Device Tree Fixup
   - 2.2 Device Tree Configuration
     - 2.2.1 Multi-Panel Node Definition
     - 2.2.2 Panel ID Configuration
   - 2.3 Driver and Device Tree Interaction Workflow
     - 2.3.1 Panel Detection and Switching Workflow
     - 2.3.2 Complete Patch
3. Debugging
   - 3.1 Key Log Analysis
   - 3.2 FDT Debugging

1. Introduction

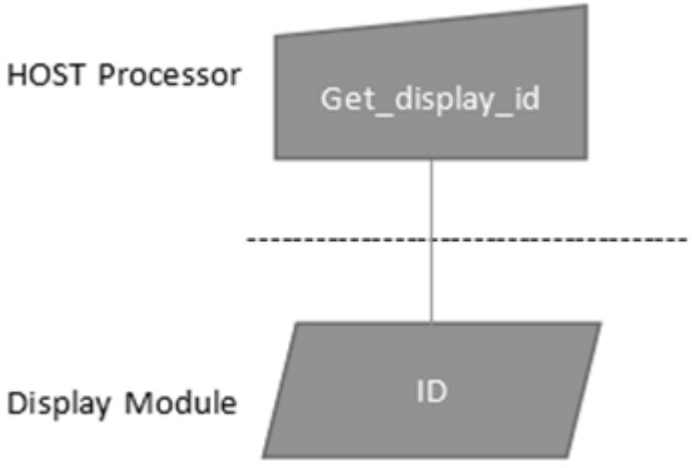
This guide aims to assist developers and users in configuring and enabling compatibility support for multiple secondary-supplier MIPI displays on Rockchip SoC platforms equipped with a MIPI DSI interface. Secondary-supplier displays refer to alternative screens from different vendors reserved for a project. This guide provides instructions on adapting the Driver layer and configuring the Device Tree to enable dynamic detection and switching among these secondary-supplier displays under a single Firmware image on the same motherboard hardware.

2. Core Functionality Description

2.1 Multi-Display Compatibility Mechanism

2.1.1 Dynamic Panel Detection

Leveraging the bidirectional transmission capability of MIPI DSI Lane0, the system reads the ID information from the connected display and automatically matches it against predefined Device Tree configuration descriptions for displays.



2.1.2 Device Tree Fixup

Before enabling the display path in the display system during the U-Boot stage, obtain the ID of the connected panel. Dynamically adjust the status of Device Tree nodes based on the matching results, activating only the Device Tree configuration (including initialization sequences and resolution information) for the matched panel.

2.2 Device Tree Configuration

2.2.1 Multi-Panel Node Definition

In the DTS, define the power-on control, power enable, RST, backlight, MIPI DSI operating mode, number of lanes, DSC enablement, DPHY or CPHY selection, panel initialization sequences, and corresponding panel resolution for all secondary-supplier panels.

```
&dsi0 {
    status = "okay";

    dsi0_panel0: panel0@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        reg = <0>;

        dsi,flags = <(MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET)>;
        dsi,format = <MIPI_DSI_FMT_RGB888>;
        dsi,lanes  = <4>;

        slice-width = <720>;
        slice-height = <65>;
        version-major = <1>;
        version-minor = <1>;
        compressed-data;

        id-reg = <0xdb>;
        id = [10];

        panel-init-sequence = [
            /* panel0 init sequence */
        ];

        panel-exit-sequence = [
            /* panel0 exit sequence */
        ];

        disp0_timings0: display-timings {
            native-mode = <&dsi_timing0>;
            dsi_timing0: timing0 {
                clock-frequency = <280000000>;
                hactive = <1440>;
                vactive = <3120>;
                hfront-porch = <16>;
                hsync-len = <8>;
                hback-porch = <8>;
                vfront-porch = <4>;
                vsync-len = <16>;
                vback-porch = <8>;
                hsync-active = <0>;
                vsync-active = <0>;
```

```
            de-active = <0>;
            pixelclk-active = <0>;
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            panel0_in_dsi0: endpoint {
                remote-endpoint = <&dsi0_out_panel>;
            };
        };
    };
};

dsi0_panel1: panel1@0 {
    status = "okay";
    compatible = "simple-panel-dsi";
    reg = <0>;

    dsi,flags = <(MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_BURST |
        MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET)>;
    dsi,format = <MIPI_DSI_FMT_RGB888>;
    dsi,lanes  = <2>;

    id-reg = <0xdb>;
    id = [72];

    panel-init-sequence = [
        /* panel1 init sequence */
    ];

    panel-exit-sequence = [
        /* panel1 exit sequence */
    ];

    disp0_timings1: display-timings {
        native-mode = <&dsi_timing1>;
        dsi_timing1: timing0 {
            clock-frequency = <110000000>;
            hactive = <1080>;
            vactive = <1920>;
            hfront-porch = <15>;
            hsync-len = <2>;
            hback-porch = <30>;
            vfront-porch = <15>;
            vsync-len = <2>;
            vback-porch = <15>;
            hsync-active = <0>;
            vsync-active = <0>;
```

```
            de-active = <0>;
            pixelclk-active = <1>;
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            panel1_in_dsi0: endpoint {
                remote-endpoint = <&dsi0_out_panel>;
            };
        };
    };
  };

  /* more panels */

  ports {
    #address-cells = <1>;
    #size-cells = <0>;

    port@1 {
      reg = <1>;
      dsi0_out_panel: endpoint {
        remote-endpoint = <&panel0_in_dsi0>;
      };
    };
  };

};
```

2.2.2 Panel ID Configuration

First, confirm with the panel vendor the commands and expected IDs for obtaining the IDs of all secondary supplier panels. Verify on the actual panel whether the IDs can be successfully retrieved as expected. Then, describe the ID parameters in the Device Tree.

Property	Function	Example Value
id-reg	Stores the panel ID Register address	`<0xdb>` (Register 0xdb; addresses generally differ across panels)
id	Expected ID value (hexadecimal array)	`[10]` (length may also vary depending on the panel)

2.3 Driver and Device Tree Interaction Workflow

2.3.1 Panel Detection and Switching Workflow

```
Start
  |
  v
[Driver Loading]
  |
  v
[Iterate through all panel nodes under the DSI controller and store the corresponding information]
  |
  v
[Pre-initialization (pre_prepare)]
  |
  |-- [Initialize MIPI DSI]
  |
  v
[ID Matching Phase]
  |
  |-- [Send panel-read-id-sequence (if defined)]
  |-- [Read id-reg value via DCS]
  |-- [Compare with the id value in the Device Tree]
  |-- [Record the index of the matched panel]
  |
  v
[Dynamic Device Tree Fixup (fixup_dts)]
  |
  |-- [Activate the remote-endpoint of the matched panel]
  |-- [Disable unmatched panel nodes: fdt_setprop_string(..., "status", "disabled")]
  |
  v
[Display Initialization]
  |
  |-- [Initialize using the timing and command sequence matching the panel]
  |
  v
End
```

Since the multi-panel compatibility logic retrieves the panel ID in advance during the U-Boot stage and iterates through all panel nodes under the &dsi node in the Device Tree, it incurs a certain overhead on boot time (primarily due to panel power-on timing delays). Furthermore, multi-panel compatibility depends on the U-Boot boot logo functionality.

2.3.2 Complete Patch

```
From ace0a1101133a65dd535043437747916c3f331dc Mon Sep 17 00:00:00 2001
From: Zhibin Huang <zhibin.huang@rock-chips.com>
Date: Sat, 23 Mar 2024 21:39:32 +0800
Subject: [PATCH] video/drm: support multi mipi screen

Type: Function
Redmine ID: N/A
Associated modifications: N/A
Test: N/A

Signed-off-by: Zhibin Huang <zhibin.huang@rock-chips.com>
Change-Id: I4abe3e8bf6a4b226bc400054fa1c0ba523aafd93
---

diff --git a/drivers/video/drm/dw_mipi_dsi.c b/drivers/video/drm/dw_mipi_dsi.c
index 2ebe826.6e13b24 100644
--- a/drivers/video/drm/dw_mipi_dsi.c
+++ b/drivers/video/drm/dw_mipi_dsi.c
@@ -18,6 +18,7 @@
 #include <syscon.h>
 #include <asm/arch-rockchip/clock.h>
 #include <linux/iopoll.h>
+#include <fdt_support.h>

 #include "rockchip_display.h"
 #include "rockchip_crtc.h"
@@ -1259,6 +1260,46 @@
     dw_mipi_dsi_pre_enable(dsi->slave);
 }

+static void dw_mipi_dsi_connector_pre_prepare(struct rockchip_connector *conn,
+                      struct display_state *state)
+{
+   struct connector_state *conn_state = &state->conn_state;
+   struct dw_mipi_dsi *dsi = dev_get_priv(conn->dev);
+   unsigned long lane_rate = 1000000000UL;
+
+   memcpy(&dsi->mode, &conn_state->mode, sizeof(struct drm_display_mode));
+   if (dsi->slave)
+      memcpy(&dsi->slave->mode, &dsi->mode,
+             sizeof(struct drm_display_mode));
+
+   if (dsi->dphy.phy)
+      dw_mipi_dsi_set_hs_clk(dsi, lane_rate);
+   else
+      dw_mipi_dsi_set_pll(dsi, lane_rate);
+
+   if (dsi->slave && dsi->slave->dphy.phy)
+      dw_mipi_dsi_set_hs_clk(dsi->slave, lane_rate);
+
```

```
+	printf("final DSI-Link bandwidth: %u Mbps x %d\n",
+	       dsi->lane_mbps, dsi->slave ? dsi->lanes * 2 : dsi->lanes);
+
+	dw_mipi_dsi_init(dsi);
+	dw_mipi_dsi_dpi_config(dsi, &dsi->mode);
+	dw_mipi_dsi_packet_handler_config(dsi);
+	dw_mipi_dsi_video_mode_config(dsi);
+	dw_mipi_dsi_command_mode_config(dsi);
+	dsi_update_bits(dsi, DSI_MODE_CFG, CMD_VIDEO_MODE, COMMAND_MODE);
+
+	dw_mipi_dsi_dphy_timing_config(dsi);
+	dw_mipi_dsi_dphy_interface_config(dsi);
+
+	dw_mipi_dsi_clear_err(dsi);
+
+	mipi_dphy_init(dsi);
+	mipi_dphy_power_on(dsi);
+	dsi_write(dsi, DSI_PWR_UP, POWERUP);
+}
+
 static int dw_mipi_dsi_connector_prepare(struct rockchip_connector *conn,
 					 struct display_state *state)
 {
@@ -1320,12 +1361,50 @@
 	return 0;
 }

+static void dw_mipi_dsi_connector_fixup_dts(struct rockchip_connector *conn,
++					    struct display_state *state)
+{
+	struct panel_state *panel_state = &state->panel_state;
+	struct rockchip_panel *panel = panel_state->panel;
+	ofnode *rockchip_panel_node = &panel->dev->node;
+	const struct device_node *conn_node;
+	char conn_endpoint_path[128], panel_endpoint_path[128];
+	const char *conn_path, *panel_path;
+	void *blob = (void*)gd->fdt_blob;
+	ofnode node;
+	u32 phandle = 0;
+	int offset;
+
+	conn_node = ofnode_to_np(conn->dev->node);
+	conn_path = conn_node->full_name;
+	strcpy(conn_endpoint_path, conn_path);
+	strcat(conn_endpoint_path, "/ports/port@1/endpoint");
+
+	panel_path = rockchip_panel_node->np->full_name;
+	strcpy(panel_endpoint_path, panel_path);
+	strcat(panel_endpoint_path, "/ports/port@0/endpoint");
+	phandle = fdt_get_phandle(blob, fdt_path_offset(blob, panel_endpoint_path));
+	do_fixup_by_path_u32(blob, (const char*)conn_endpoint_path,
++			     "remote-endpoint", phandle, 1);
+
```

```
+   ofnode_for_each_subnode(node, conn->dev->node) {
+       if (strstr(node.np->name, "panel") &&
+           ofnode_valid(node) && ofnode_is_available(node) &&
+           strcmp(node.np->full_name, panel_path)) {
+           offset = fdt_path_offset(blob, node.np->full_name);
+           fdt_setprop_string(blob, offset, "status", "disa");
+       }
+   }
+}
+
 static const struct rockchip_connector_funcs dw_mipi_dsi_connector_funcs = {
    .init = dw_mipi_dsi_connector_init,
    .prepare = dw_mipi_dsi_connector_prepare,
    .unprepare = dw_mipi_dsi_connector_unprepare,
    .enable = dw_mipi_dsi_connector_enable,
    .disable = dw_mipi_dsi_connector_disable,
+   .pre_prepare = dw_mipi_dsi_connector_pre_prepare,
+   .fixup_dts = dw_mipi_dsi_connector_fixup_dts,
 };

 static int dw_mipi_dsi_probe(struct udevice *dev)
diff --git a/drivers/video/drm/dw_mipi_dsi2.c b/drivers/video/drm/dw_mipi_dsi2.c
index ecd086d7d62.b62fafae370 100644
--- a/drivers/video/drm/dw_mipi_dsi2.c
+++ b/drivers/video/drm/dw_mipi_dsi2.c
@@ -555,6 +555,8 @@ static void dw_mipi_dsi2_ipi_color_coding_cfg(struct dw_mipi_dsi2 *dsi2)

    if (dsi2->dsc_enable)
        grf_field_write(dsi2, IPI_FORMAT, IPI_FORMAT_DSC);
+   else
+       grf_field_write(dsi2, IPI_FORMAT, IPI_FORMAT_RGB);
 }

 static void dw_mipi_dsi2_ipi_set(struct dw_mipi_dsi2 *dsi2)
@@ -719,11 +721,13 @@ static void dw_mipi_dsi2_disable(struct dw_mipi_dsi2 *dsi2)
        dw_mipi_dsi2_disable(dsi2->slave);
 }

+static void dw_mipi_dsi2_host_softrst(struct dw_mipi_dsi2 *dsi2);
 static void dw_mipi_dsi2_post_disable(struct dw_mipi_dsi2 *dsi2)
 {
    if (!dsi2->prepared)
        return;

+   dw_mipi_dsi2_host_softrst(dsi2);
    dsi_write(dsi2, DSI2_PWR_UP, RESET);

    if (dsi2->dcphy.phy)
@@ -853,6 +857,9 @@ static int dw_mipi_dsi2_connector_init(struct rockchip_connector *conn, struct d
    if (!(dsi2->mode_flags & MIPI_DSI_MODE_VIDEO)) {
        conn_state->output_flags |= ROCKCHIP_OUTPUT_MIPI_DS_MODE;
        conn_state->hold_mode = dsi2->disable_hold_mode ? false : true;
+   } else {
```

```
+       conn_state->output_flags &= ~ROCKCHIP_OUTPUT_MIPI_DS_MODE;
+       conn_state->hold_mode = false;
    }

    if (dsi2->dual_channel) {
@@ -912,6 +919,8 @@ static int dw_mipi_dsi2_connector_init(struct rockchip_connector *conn, struct d
        cstate->dsc_sink_cap.target_bits_per_pixel_x16 = 8 << 4;
        cstate->dsc_sink_cap.native_420 = 0;
        memcpy(&cstate->pps, dsi2->pps, sizeof(struct drm_dsc_picture_parameter_set));
+   } else {
+       cstate->dsc_enable = 0;
    }

    return 0;
@@ -1185,6 +1194,31 @@ static void dw_mipi_dsi2_get_mipi_pixel_clk(struct dw_mipi_dsi2 *dsi2,
        dsi2->slave->mipi_pixel_rate = dsi2->mipi_pixel_rate;
}

+static void dw_mipi_dsi2_connector_pre_prepare(struct rockchip_connector *conn,
+                       struct display_state *state)
+{
+   struct dw_mipi_dsi2 *dsi2 = dev_get_priv(conn->dev);
+   struct connector_state *conn_state = &state->conn_state;
+   unsigned long lane_rate = 1000000000UL;
+
+   memcpy(&dsi2->mode, &conn_state->mode, sizeof(struct drm_display_mode));
+   if (dsi2->slave)
+       memcpy(&dsi2->slave->mode, &dsi2->mode,
+              sizeof(struct drm_display_mode));
+
+   if (dsi2->dcphy.phy)
+       dw_mipi_dsi2_set_hs_clk(dsi2, lane_rate);
+
+   if (dsi2->slave && dsi2->slave->dcphy.phy)
+       dw_mipi_dsi2_set_hs_clk(dsi2->slave, lane_rate);
+
+   printf("final DSI-Link bandwidth: %u %s x %d\n",
+          dsi2->lane_hs_rate, dsi2->c_option ? "Ksps" : "Kbps",
+          dsi2->slave ? dsi2->lanes * 2 : dsi2->lanes);
+
+   dw_mipi_dsi2_pre_enable(dsi2);
+}
+
static int dw_mipi_dsi2_connector_prepare(struct rockchip_connector *conn,
                      struct display_state *state)
{
@@ -1281,6 +1315,42 @@ static int dw_mipi_dsi2_connector_mode_valid(struct rockchip_connector *conn,
    return MODE_OK;
}

+static void dw_mipi_dsi2_connector_fixup_dts(struct rockchip_connector *conn,
+                       struct display_state *state)
+{
```

```
+   struct panel_state *panel_state = &state->panel_state;
+   struct rockchip_panel *panel = panel_state->panel;
+   ofnode *rockchip_panel_node = &panel->dev->node;
+   const struct device_node *conn_node;
+   char conn_endpoint_path[128], panel_endpoint_path[128];
+   const char *conn_path, *panel_path;
+   void *blob = (void*)gd->fdt_blob;
+   ofnode node;
+   u32 phandle = 0;
+   int offset;
+
+   conn_node = ofnode_to_np(conn->dev->node);
+   conn_path = conn_node->full_name;
+   strcpy(conn_endpoint_path, conn_path);
+   strcat(conn_endpoint_path, "/ports/port@1/endpoint");
+
+   panel_path = rockchip_panel_node->np->full_name;
+   strcpy(panel_endpoint_path, panel_path);
+   strcat(panel_endpoint_path, "/ports/port@0/endpoint");
+   phandle = fdt_get_phandle(blob, fdt_path_offset(blob, panel_endpoint_path));
+   do_fixup_by_path_u32(blob, (const char*)conn_endpoint_path,
+               "remote-endpoint", phandle, 1);
+
+   ofnode_for_each_subnode(node, conn->dev->node) {
+       if (strstr(node.np->name, "panel") &&
+           ofnode_valid(node) && ofnode_is_available(node) &&
+           strcmp(node.np->full_name, panel_path)) {
+           offset = fdt_path_offset(blob, node.np->full_name);
+           fdt_setprop_string(blob, offset, "status", "disa");
+       }
+   }
+}
+
static const struct rockchip_connector_funcs dw_mipi_dsi2_connector_funcs = {
    .pre_init = dw_mipi_dsi2_connector_pre_init,
    .init = dw_mipi_dsi2_connector_init,
@@ -1289,6 +1359,8 @@ static const struct rockchip_connector_funcs dw_mipi_dsi2_connector_funcs = {
    .enable = dw_mipi_dsi2_connector_enable,
    .disable = dw_mipi_dsi2_connector_disable,
    .mode_valid = dw_mipi_dsi2_connector_mode_valid,
+   .pre_prepare = dw_mipi_dsi2_connector_pre_prepare,
+   .fixup_dts = dw_mipi_dsi2_connector_fixup_dts,
 };

static int dw_mipi_dsi2_probe(struct udevice *dev)
diff --git a/drivers/video/drm/rockchip_connector.h b/drivers/video/drm/rockchip_connector.h
index 8c2ff75.e098b7f 100644
--- a/drivers/video/drm/rockchip_connector.h
+++ b/drivers/video/drm/rockchip_connector.h
@@ -183,6 +183,12 @@
     */
    int (*get_edid)(struct rockchip_connector *connector, struct display_state *state);
    /*
```

```
+    * for reading panel id, configure mipi host and phy simply
+    * so that reading and writing commands can send successfully.
+    */
+   void (*pre_prepare)(struct rockchip_connector *connector, struct display_state *state);
+   void (*fixup_dts)(struct rockchip_connector *connector, struct display_state *state);
+   /*
     * call before crtc enable.
     */
    int (*prepare)(struct rockchip_connector *connector, struct display_state *state);
diff --git a/drivers/video/drm/rockchip_display.c b/drivers/video/drm/rockchip_display.c
index ae99b4d.1042f41 100644
--- a/drivers/video/drm/rockchip_display.c
+++ b/drivers/video/drm/rockchip_display.c
@@ -762,6 +762,15 @@
    if (!ret && !state->force_output)
       goto deinit;

+   if (conn->panel) {
+      rockchip_panel_pre_prepare(conn->panel);
+      if (state->enabled_at_spl == false) {
+         ret = rockchip_connector_init(state);
+         if (ret)
+            goto deinit;
+      }
+   }
+
    ret = 0;
    if (state->enabled_at_spl == true) {
 #ifdef CONFIG_SPL_BUILD
@@ -1736,12 +1745,17 @@
    int ret = 0;

    *panel = NULL;
-   panel_ofnode = dev_read_subnode(dev, "panel");
-   if (ofnode_valid(panel_ofnode) && ofnode_is_available(panel_ofnode)) {
-      ret = uclass_get_device_by_ofnode(UCLASS_PANEL, panel_ofnode,
-                        &panel_dev);
-      if (!ret)
-         goto found;
+   ofnode_for_each_subnode(panel_ofnode, dev->node) {
+      if (!strstr(panel_ofnode.np->name, "panel"))
+         continue;
+      if (ofnode_valid(panel_ofnode) &&
+         ofnode_is_available(panel_ofnode)) {
+         ret = uclass_get_device_by_ofnode(UCLASS_PANEL,
+                           panel_ofnode,
+                           &panel_dev);
+         if (!ret)
+            goto found;
+      }
    }

    ep_node = rockchip_of_graph_get_remote_node(dev->node, PORT_DIR_OUT, 0);
```

```
diff --git a/drivers/video/drm/rockchip_panel.c b/drivers/video/drm/rockchip_panel.c
index 9ddb4d8.f4b91c5 100644
--- a/drivers/video/drm/rockchip_panel.c
+++ b/drivers/video/drm/rockchip_panel.c
@@ -15,16 +15,20 @@
 #include <spi.h>
 #include <asm/gpio.h>
 #include <dm/device.h>
+#include <dm/device-internal.h>
 #include <dm/read.h>
 #include <dm/uclass.h>
 #include <dm/uclass-id.h>
 #include <linux/media-bus-format.h>
 #include <power/regulator.h>
+#include <dm/of_access.h>
+#include <fdt_support.h>

 #include "rockchip_display.h"
 #include "rockchip_crtc.h"
 #include "rockchip_connector.h"
 #include "rockchip_panel.h"
+#include "rockchip_phy.h"

 struct rockchip_cmd_header {
   u8 data_type;
@@ -42,6 +46,11 @@
   int cmd_cnt;
 };

+struct dsi_panel_id {
+   u8 *buf;
+   int len;
+};
+
 struct rockchip_panel_plat {
   bool power_invert;
   u32 bus_format;
@@ -58,6 +67,10 @@

   struct rockchip_panel_cmds *on_cmds;
   struct rockchip_panel_cmds *off_cmds;
+   struct rockchip_panel_cmds *read_id_cmds;
+
+   unsigned int id_reg;
+   struct dsi_panel_id *id;
 };

 struct rockchip_panel_priv {
@@ -411,7 +424,205 @@
   priv->enabled = false;
 }

+static int panel_simple_dsi_match_id(struct rockchip_panel *panel)
```

```
+{
+	struct rockchip_panel_plat *plat = dev_get_platdata(panel->dev);
+	struct rockchip_panel_priv *priv = dev_get_priv(panel->dev);
+	struct mipi_dsi_device *dsi = dev_get_parent_platdata(panel->dev);
+	int cmd_len = plat->id->len;
+	int read_count = 1, match_count;
+	int ret = -EINVAL;
+	u8 *panel_id;
+
+	panel_id = devm_kzalloc(panel->dev, sizeof(char) * cmd_len, GFP_KERNEL);
+	if (!panel_id)
+		return -ENOMEM;
+
+	if (priv->power_supply)
+		regulator_set_enable(priv->power_supply, !plat->power_invert);
+
+	if (dm_gpio_is_valid(&priv->enable_gpio))
+		dm_gpio_set_value(&priv->enable_gpio, 1);
+
+	if (plat->delay.prepare)
+		mdelay(plat->delay.prepare);
+
+	if (dm_gpio_is_valid(&priv->reset_gpio))
+		dm_gpio_set_value(&priv->reset_gpio, 1);
+
+	if (plat->delay.reset)
+		mdelay(plat->delay.reset);
+
+	if (dm_gpio_is_valid(&priv->reset_gpio))
+		dm_gpio_set_value(&priv->reset_gpio, 0);
+
+	if (plat->delay.init)
+		mdelay(plat->delay.init);
+
+	if (plat->read_id_cmds) {
+		rockchip_panel_send_dsi_cmds(dsi, plat->read_id_cmds);
+		free(plat->read_id_cmds);
+		plat->read_id_cmds = NULL;
+	}
+
+	if (cmd_len > 1)
+		mipi_dsi_set_maximum_return_packet_size(dsi, cmd_len);
+
+	for (int i = 0; i < read_count; i++) {
+
+		printf("panel id, reg: 0x%02x, val: ", plat->id_reg);
+		for (int j = 0; j < cmd_len; j++)
+			printf("%02x", plat->id->buf[j]);
+		printf("\n");
+		ret = mipi_dsi_dcs_read(dsi, plat->id_reg, panel_id, cmd_len);
+		if (ret > 0) {
+			printf("read panel id, reg: 0x%02x, val: ", plat->id_reg);
+			match_count = 0;
```

```
+		for (int j = 0; j < cmd_len; j++) {
+			printf("%02x", panel_id[j]);
+			if (panel_id[j] == plat->id->buf[j])
+				match_count++;
+		}
+		printf("\n");
+		if (match_count == cmd_len) {
+			ret = 0;
+			break;
+		}
+		ret = -EINVAL;
+	} else {
+		printf("read panel timeout or no response\n");
+		ret = -ETIMEDOUT;
+	}
+  }
+
+  if (dm_gpio_is_valid(&priv->reset_gpio))
+	dm_gpio_set_value(&priv->reset_gpio, 1);
+
+  if (dm_gpio_is_valid(&priv->enable_gpio))
+	dm_gpio_set_value(&priv->enable_gpio, 0);
+
+  if (priv->power_supply)
+	regulator_set_enable(priv->power_supply, plat->power_invert);
+
+  if (plat->delay.unprepare)
+	mdelay(plat->delay.unprepare);
+
+  return ret;
+}
+
+static int panel_simple_dsi_match(struct display_state *state, int panel_num)
+{
+  struct connector_state *conn_state = &state->conn_state;
+  struct panel_state *panel_state = &state->panel_state;
+  struct rockchip_connector *conn = conn_state->connector;
+  const struct rockchip_connector_funcs *conn_funcs = conn->funcs;
+  struct rockchip_panel *panel;
+  ofnode panel_ofnode;
+  struct udevice *panel_dev;
+  int ret;
+
+  if (conn_funcs->pre_init) {
+	ret = conn_funcs->pre_init(conn, state);
+	if (ret) {
+		printf("failed to pre_init connector\n");
+		goto deinit;
+	}
+  }
+
+  if (conn_funcs->init) {
+	ret = conn_funcs->init(conn, state);
```

```
+       if (ret) {
+           printf("failed to init connector\n");
+           goto deinit;
+       }
+   }
+
+   if (conn->phy)
+       rockchip_phy_init(conn->phy);
+
+   if (conn_funcs->pre_prepare)
+       conn_funcs->pre_prepare(conn, state);
+
+   ofnode_for_each_subnode(panel_ofnode, conn->dev->node) {
+       if (!strstr(panel_ofnode.np->name, "panel"))
+           continue;
+       if (ofnode_valid(panel_ofnode) &&
+           ofnode_is_available(panel_ofnode)) {
+           ret = uclass_get_device_by_ofnode(UCLASS_PANEL,
+                       panel_ofnode,
+                       &panel_dev);
+           if (ret) {
+               printf("this subnode is not a panel node\n");
+               continue;
+           }
+       } else {
+           continue;
+       }
+
+       panel = (struct rockchip_panel *)dev_get_driver_data(panel_dev);
+       if (!panel) {
+           printf("cannot found panel\n");
+           ret = -ENODEV;
+           break;
+       }
+
+       panel_state->panel = panel;
+       conn->panel = panel;
+       panel->state = state;
+
+       printf("try to match %s\n", panel_ofnode.np->full_name);
+       ret = panel_simple_dsi_match_id(panel);
+       if (!ret)
+           break;
+
+       if (panel_num > 1) {
+           device_remove(panel_dev, DM_REMOVE_NORMAL);
+           panel_num--;
+       }
+   }
+
+   if (conn_funcs->disable)
+       conn_funcs->disable(conn, state);
+
```

```
+   if (conn_funcs->unprepare)
+       conn_funcs->unprepare(conn, state);
+
+   if (conn_funcs->fixup_dts)
+       conn_funcs->fixup_dts(conn, state);
+
+deinit:
+   if (conn_funcs->deinit)
+       conn_funcs->deinit(conn, state);
+
+   return ret;
+}
+
+static void panel_simple_pre_prepare(struct rockchip_panel *panel)
+{
+   struct display_state *state = panel->state;
+   struct connector_state *conn_state = &state->conn_state;
+   struct rockchip_connector *conn = conn_state->connector;
+   char *conn_name = (char *)conn->dev->node.np->full_name;
+   ofnode panel_ofnode;
+   int panel_num = 0;
+
+   conn_state->bus_format = panel->bus_format;
+
+   ofnode_for_each_subnode(panel_ofnode, conn->dev->node) {
+       if (strstr(panel_ofnode.np->name, "panel") &&
+           ofnode_valid(panel_ofnode) &&
+           ofnode_is_available(panel_ofnode))
+           panel_num++;
+   }
+
+   if (panel_num > 1 && strstr(conn_name, "dsi")) {
+       if (panel_simple_dsi_match(state, panel_num))
+           printf("fail to match valid panel\n");
+       else
+           printf("success to match valid panel\n");
+   }
+}
+
 static const struct rockchip_panel_funcs rockchip_panel_funcs = {
+   .pre_prepare = panel_simple_pre_prepare,
    .prepare = panel_simple_prepare,
    .unprepare = panel_simple_unprepare,
    .enable = panel_simple_enable,
@@ -436,6 +647,17 @@

    plat->bus_format = dev_read_u32_default(dev, "bus-format",
                    MEDIA_BUS_FMT_RBG888_1X24);
+   plat->id_reg = dev_read_u32_default(dev, "id-reg", 0);
+   data = dev_read_prop(dev, "id", &len);
+
+   if (data) {
+       plat->id = devm_kzalloc(dev, sizeof(*plat->id), GFP_KERNEL);
```

```
+       if (!plat->id)
+           return -ENOMEM;
+
+       plat->id->buf = devm_kzalloc(dev, sizeof(char) * len, GFP_KERNEL);
+       if (!plat->id->buf)
+           return -ENOMEM;
+
+       memcpy(plat->id->buf, data, len);
+       plat->id->len = len;
+   }
    plat->bpc = dev_read_u32_default(dev, "bpc", 8);

    data = dev_read_prop(dev, "panel-init-sequence", &len);
@@ -451,6 +673,19 @@
        }
    }

+   data = dev_read_prop(dev, "panel-read-id-sequence", &len);
+   if (data) {
+       plat->read_id_cmds = calloc(1, sizeof(*plat->read_id_cmds));
+       if (!plat->read_id_cmds)
+           return -ENOMEM;
+
+       ret = rockchip_panel_parse_cmds(data, len, plat->read_id_cmds);
+       if (ret) {
+           printf("failed to parse panel-read-id-sequence\n");
+           goto free_read_id_cmds;
+       }
+   }
+
    data = dev_read_prop(dev, "panel-exit-sequence", &len);
    if (data) {
        plat->off_cmds = calloc(1, sizeof(*plat->off_cmds));
@@ -470,6 +705,8 @@

 free_cmds:
    free(plat->off_cmds);
+free_read_id_cmds:
+   free(plat->read_id_cmds);
 free_on_cmds:
    free(plat->on_cmds);
    return ret;
@@ -574,6 +811,19 @@
    return 0;
 }

+static int panel_simple_remove(struct udevice *dev) {
+   struct rockchip_panel_priv *priv = dev_get_priv(dev);
+
+   printf("remove %s\n", dev->node.np->full_name);
+
+   if (dm_gpio_is_valid(&priv->enable_gpio))
+       dm_gpio_free(dev, &priv->enable_gpio);
```

```
+	if (dm_gpio_is_valid(&priv->reset_gpio))
+		dm_gpio_free(dev, &priv->reset_gpio);
+
+	return 0;
+}
+
 static const struct udevice_id rockchip_panel_ids[] = {
 	{ .compatible = "simple-panel", },
 	{ .compatible = "simple-panel-dsi", },
@@ -587,6 +837,7 @@
 	.of_match = rockchip_panel_ids,
 	.ofdata_to_platdata = rockchip_panel_ofdata_to_platdata,
 	.probe = rockchip_panel_probe,
+	.remove = panel_simple_remove,
 	.priv_auto_alloc_size = sizeof(struct rockchip_panel_priv),
 	.platdata_auto_alloc_size = sizeof(struct rockchip_panel_plat),
 };
diff --git a/drivers/video/drm/rockchip_panel.h b/drivers/video/drm/rockchip_panel.h
index 5e49fe3.b503e84 100644
--- a/drivers/video/drm/rockchip_panel.h
+++ b/drivers/video/drm/rockchip_panel.h
@@ -12,10 +12,12 @@
 struct rockchip_conn;

 struct rockchip_panel_funcs {
+	void (*pre_prepare)(struct rockchip_panel *panel);
 	void (*prepare)(struct rockchip_panel *panel);
 	void (*unprepare)(struct rockchip_panel *panel);
 	void (*enable)(struct rockchip_panel *panel);
 	void (*disable)(struct rockchip_panel *panel);
+	void (*init)(struct rockchip_panel *panel);
 	int (*get_mode)(struct rockchip_panel *panel,
 			struct drm_display_mode *mode);
 };
@@ -48,6 +50,15 @@
 	state->conn_state.bpc = panel->bpc;
 }

+static inline void rockchip_panel_pre_prepare(struct rockchip_panel *panel)
+{
+	if (!panel)
+		return;
+
+	if (panel->funcs && panel->funcs->pre_prepare)
+		panel->funcs->pre_prepare(panel);
+}
+
 static inline void rockchip_panel_prepare(struct rockchip_panel *panel)
 {
 	if (!panel)
```

3. Debugging

3.1 Key Log Analysis

Investigate the cause of screen ID acquisition failure based on the following:

```
# ID Match Success Log
read panel id, reg: 0xDA, val: 09      # Read ID Value
success to match valid panel          # Driver Confirmation and Matching

# Exception Scenarios
fail to read panel id                 # Communication Failure
ID mismatch: expect 08 got 09         # ID Mismatch
```

3.2 FDT Debugging

```
adb pull /sys/firmware/fdt ./fdt

Execute DTB decompilation in the kernel root directory.
./scripts/dtc/dtc -I dtb -O dts -o fdt.dts ./fdt
```

Check fdt.dts to verify whether the node status matches expectations, for example:

```
panel0@0 { status = "okay"; ... };
panel1@0 { status = "disabled"; ... };
```